

Chapter 1:

Interprocedural Parallelization Analysis: A Case Study

Mary W. Hall * Brian R. Murphy * Saman P. Amarasinghe *

Abstract

We present an overview of our *interprocedural analysis* system, which applies the program analysis required for parallelization across procedure boundaries. We discuss the issues we addressed to efficiently obtain precise results in the interprocedural setting. We present the analysis required for parallelization, illustrated with an excerpt from a Fortran benchmark program. By integrating a comprehensive suite of interprocedural analyses, we have built a system that is much more effective at locating parallelism in scientific benchmarks than earlier interprocedural systems.

1 Introduction

A key performance concern in automatically parallelizing a program is locating *coarse-grain parallelism*, independent computations that perform a significant amount of work. Coarse-grain parallel computations exhibit very small overhead costs from synchronization and parallel thread initiation relative to the amount of time spent doing useful work in parallel. Overhead costs can be significant; without sufficient computation in parallel regions of the program, speedups are unlikely, and performance degradations may even result.

Modular programs often contain coarse-grain parallel computations that span multiple procedures in the program. Most compilers will fail to parallelize these computations because they analyze each procedure as an independent unit. They must make conservative assumptions about what effects a called procedure will have on the data it accesses. Ideally, a compiler should be just as effective across procedure boundaries as within a single procedure, so that a programmer is not penalized by the compiler for writing in a modular style. To achieve this goal, the compiler must perform *interprocedural analysis*, analysis over the whole program.

Although there has been much research in the area of interprocedural analysis for parallelization, the results to date have been inconclusive and sophisticated interprocedural analysis has not been adopted in practice. Interprocedural compilers are significantly harder to build than their intraprocedural counterparts. The compiler must be fundamentally restructured to represent the interprocedural structure of the program, handling program features not encountered in an intraprocedural setting such as parameter passing. Further, implementation of individual analyses is more difficult for two reasons. First, techniques that are practical within a single procedure may be prohibitively expensive when performed across the whole program; new approaches are required to make the analysis practical without sacrificing precision. Second, in the presence of obtuse program constructs,

*Computer Systems Laboratory, Stanford University, Stanford, CA 94305-4070
This research was supported in part by ARPA contracts N00039-91-C-0138 and DABT63-91-K-0003, an NSF CISE postdoctoral fellowship, a fellowship from Intel Corporation, and a fellowship from AT&T Bell Laboratories.

intraprocedural analyses may make simplifying approximations that occur rarely enough to not significantly hurt precision. When we attempt interprocedural parallelization of coarse-grain loops consisting of thousands of lines of code, however, these approximations will be performed much more frequently and precision will be lost. An interprocedural analysis should avoid such approximations whenever feasible.

Automatic parallelizers typically identify loops as their main source of coarse-grain parallelism. To test if a loop can be parallelized, the compiler must determine whether executing the loop iterations in parallel preserves the sequential ordering of writes to a memory location (relative to the reads and other writes of that location). This analysis must consider accesses to both scalar variables (*scalar data-flow analysis*) and arrays (*array data-flow analysis*). Scalar data-flow analysis must also assume the more difficult role of determining values of subscript expressions and loop bounds to assist array analysis.

This paper presents an overview of an interprocedural parallelization analysis system. The next section describes issues in the interprocedural framework. The subsequent section overviews the scalar and array analyses. The final section highlights results we have gathered with this system.

2 Interprocedural Framework

Interprocedural parallelization depends upon the solution of a large number of interprocedural data-flow analysis problems. These problems share many commonalities. We have encapsulated these common features in a tool, FIAT [1], which we have combined with the Stanford SUIF compiler to constitute our interprocedural parallelization system. FIAT is an *interprocedural framework*, analogous to traditional data-flow analysis frameworks [4]. A framework is even more important for interprocedural optimization because of the complexity of collecting and managing information about all the procedures in a program. To facilitate rapid system prototyping, FIAT provides *parameterized templates* for interprocedural program analysis which encapsulate common analysis features (an example of such a template is the interval-style analysis described in Section 2.2) A user of a template instantiates it with a collection of functions unique to their particular analysis requirements.

The remainder of this section describes how the system manages the costs of interprocedural analysis without giving up precision in the analysis.

2.1 Selective Procedure Cloning

For procedures invoked on multiple distinct paths through a program, traditional interprocedural program analysis forms a conservative approximation of the information entering the procedure that is correct for both paths. Such approximations can affect the precision of analysis if a procedure is invoked along paths that contribute very different information.

To illustrate the effects of path-specific information on optimization, consider the following example taken from the Spec89 benchmark **matrix300**.

```
SUBROUTINE SAXPY(...,X,IX,Y,IY)
  REAL X(IX,1), Y(IY,1)
  DO I = 1, N
    Y(1,I) = Y(1,I) + A*X(1,I)
```

The arrays **X** and **Y** have symbolic dimension sizes, inhibiting any optimizations that rely on precise knowledge of array accesses. In calls to **SAXPY**, the value passed to **IX** is either 1 or 100, depending on whether we are accessing **X** or its transpose. **IY** is similarly either 1 or 102. Because their values vary across invocations, traditional techniques assume

no knowledge of the values of these dimension variables, resulting in lost precision.

One way to obtain path-specific information is by applying *inline substitution*, whereby the compiler replaces a procedure call with the body of the called procedure. Unfortunately, full inlining often leads to unmanageable code explosion.

An ideal approach would utilize path-specific information to obtain the precision of inlining only when it provides opportunities for optimization. For this reason, we incorporate *selective procedure cloning*, a program restructuring in which the compiler replicates a procedure to optimize it in the context of distinct calling environments. By applying cloning *selectively* according to the information it exposes, we can obtain the same information as full inlining without unnecessarily replicating procedures along all paths.

2.2 Interprocedural Interval Analysis

Straightforward adaptation of intraprocedural analysis techniques to the interprocedural setting may result in a system that is slow to converge to a solution. This efficiency problem arises because values flow both within each procedure and between a procedure and its callers. By separating calling context from side effects, we can perform analysis efficiently in two passes through the program. A bottom-up analysis (*i.e.*, procedures are analyzed before their callers) produces descriptions of the *behavior* of a subroutine; in a top-down analysis, calling context is applied to these behavior descriptions to derive the final analysis results for a procedure. Within each procedure, we aggregate information at loop boundaries; this corresponds to what is traditionally called interval-based analysis. Most of the analyses described in the next section are performed in this way.

3 Parallelization Analysis

This section describes the analysis performed by our parallelizer and illustrates each of these with an excerpt from the Perfect benchmark program, **spec77**. This program is a spectral analysis code; the outer loop of the program is a time step loop, which invokes similar subroutines **GLOOP** and **GWATER**. The outer loops of these subroutines are parallelizable, providing a good source of coarse-grain parallelism. Figure 1 shows the outer loop of **GLOOP**, which is a good illustration of our system because it requires a collection of analyses in order to be parallelized. This loop consists of over 5000 lines of Fortran code.

3.1 Analysis of Scalar Variables

A number of standard analyses ensure that scalar variables do not limit the parallelism available in a loop. These include scalar dependence testing, scalar privatization, and detection of induction variables and reductions. These basic techniques are well known; using FIAT's templates, applying them interprocedurally is straightforward.

In addition to these basic techniques are the scalar analyses needed to support precise analysis of arrays; these must track the *values* of scalar variables, to determine which elements of an array might be accessed by an array reference. To do this we perform an analysis which unifies a host of standard intraprocedural scalar data-flow analyses interprocedurally; these include constant propagation, detection of induction and loop-invariant variables, and common subexpression elimination.

To illustrate the utility of these supporting scalar analyses, consider the subroutine **FFS99** from the example above. To determine which elements of array **W** may be written in a particular iteration of the loops, we need to determine the values of **I** and **J**. Our analysis determines that on iteration number **M** of the outer loop and iteration **N** of the inner loop,

```

SUBROUTINE GLOOP()
  COMPLEX PLN(961)
  REAL TF(96,12)
  // we would like to parallelize this loop!
  DO LAT = 1, 38
    ...
    CALL GFIDI()
    DO K = 1, 12
      CALL FL22(TF(1,K),...,Y(1,K),PLN)
    ...
  SUBROUTINE FL22(FP,...,FLN,PLN)
    COMPLEX FP(31), PLN(31,31)
    DO LL = 1, 31
      DO I = 1, 31, 2
        FLN(I,LL) = FLN(I,LL) + ...
      DO I = 2, 30, 2
        FLN(I,LL) = FLN(I,LL) + ...
  SUBROUTINE GFIDI()
  // thirteen calls interspersed with code
  CALL FFS99()
  // nine calls interspersed with code
  CALL FFA99()

SUBROUTINE FFS99(A,W,LOT)
  ...
  IBASE=3
  JBASE=95
  DO K = 99,131,2
    I = IBASE
    J = JBASE
    DO L = 1, LOT
      W(I) = A(I) ...
      W(J) = A(I) ...
      W(I+1) = ...
      W(J+1) = ...
      I = I+96
      J = J+96
    IBASE = IBASE+2
    JBASE = JBASE+2
  // three additional loops write
  // W[1:2,1:LOT]
  // W[37:48,1:LOT],W[51:62,1:LOT]
  // W[49:50,1:LOT]
  // subsequent loops in FFS99 read
  // W[1:96,1:LOT]

```

FIG. 1. *spec77 excerpt*

the value of I is $3+2*M+96*N$ and the value of J is $95-2*M+96*N$. In effect, we substitute the derived values in place, and reduce the loops to the following, more easily analyzed form:

```

SUBROUTINE FFS99(A,W,LOT)
  ...
  DO M = 0,16
    DO N = 0, LOT-1
      W(3+2*M,N+1) = ... // writes W [3:35:2, 1:LOT]
      W(4+2*M,N+1) = ... // writes W [4:36:2, 1:LOT]
      W(95-2*M,N+1) = ... // writes W [63:95:2, 1:LOT]
      W(96-2*M,N+1) = ... // writes W [64:96:2, 1:LOT]

```

In general, for each variable of interest within a program region (loop, loop body, or procedure), the unified scalar analysis algorithm determines a value for that variable as a combination of region-invariant values: constants, iteration numbers of enclosing loops, region invariant variables, and region-invariant values of changing variables (*e.g.*, the value of a variable on entry to the region). If a variable's value cannot be expressed in these terms it is considered *unknown*. In this way we perform loop-based constant propagation, induction and loop-invariant variable detection, and common subexpression elimination as needed to derive precise scalar value information.

Although the above example does not require interprocedural analysis, the algorithm supports loop bodies spanning multiple procedures. Because a single reference may have several dissimilar immediately enclosing loops, procedure cloning can be used to reduce the imprecision that might otherwise be introduced.

3.2 Data Dependence

The basic array analysis in parallelizing a loop is data dependence analysis. We say that two accesses are data dependent if on any two iterations the two accesses refer to the same memory location. Standard data dependence analysis only applies to array accesses whose

index functions and enclosing loop bounds are affine expressions of enclosing loop indices and loop invariants. Within this domain, the data dependence problem has been shown to be equivalent to integer programming, a potentially expensive problem. However, the data dependence problems found in practice are simple, and efficient algorithms have been developed to solve these problems exactly.

By rewriting the loop bounds and array indices as linear functions of outer loop indices, the scalar analysis phase makes standard data dependence analysis more often applicable. For example, while none of the accesses in the **FFS99** subroutine in the original program is in this form, all of the array indices in the rewritten code are affine expressions, allowing the determination that both the **M** and **N** loops are parallelizable.

3.3 Array Summaries

Traditional data dependence analysis solves an integer programming problem for every pair of array accesses in a loop of interest. This $O(n^2)$ analysis becomes prohibitively expensive for very large loops. One way to improve efficiency is to summarize the array accesses in a region of code; a data dependence analysis is then applied to a small number of summaries.

A set of array accesses is represented by a set of linear inequalities: the array indices are equated to affine expressions of outer loop indices and loop-invariant values, constrained further by inequalities derived from the loop bounds. We create a summary of an access outside the enclosing loop by projecting away the loop index variable.

For example, the access **W**(3+2*M, **N**) within the **M** and **N** loops is summarized as

$$\left\{ (a_1, a_2) \left| \begin{array}{ll} a_1 = 3 + 2 * M, & 0 \leq M \leq 16 \\ a_2 = N, & 0 \leq N \leq LOT - 1 \end{array} \right. \right\}$$

Combining the summaries of the first four loops, the compiler can determine that the subroutine **FFS99** writes the entire array **W**.

3.4 Array Privatization

A common practice in Fortran codes is the use of array variables as temporaries. Data dependences on such arrays are often simply the result of reusing storage; no values flow beyond the current iteration of the loop. If a private copy of the array is created for each parallel process, the loop can be executed in parallel.

In the outer loop of **GLOOP**, **W** is a work array that is written every iteration before it is accessed. The loop is not directly parallelizable. However, by recognizing that all the reads to the array **W** follow writes within the same iteration of the loop, the compiler determines that the **W** array is privatizable, and that the **LAT** loop can be parallelized.

3.5 Reductions

Reductions are computations using associative operations. By relaxing the ordering constraint on the operations, we can exploit more parallelism than using just the standard dependence and privatization tests. A simple extension to the array analysis detects interprocedural reductions; for several programs it has revealed additional parallelism.

For example, in the outer (**LAT**) loop of **GLOOP**, there is a reduction on variable **Y**, but the associative operations involved in the reduction appear as operations on array **FLN** in subroutine **FL22**. We recognize the reduction as follows: First, we discover the associative operations in **FL22** and mark the summaries of accesses of **FLN** as potential reductions. Second, we evaluate whether the loops are parallel, from innermost outward, renaming **FLN**

to **Y** at the call. Finally, when considering the **LAT** loop, we see that the loop carries a dependence on **Y**, but that it was marked as a potential reduction. Thus we can generate special reduction code and parallelize the loop.

3.6 Array Reshapes

Array reshaping occurs when the same array is accessed in different procedures as an array of different dimensions. In our example, two common uses are illustrated. Array **TF**, a two-dimensional array of reals, is passed a column at a time to **FL22**, where the column is viewed as a one-dimensional array of complex variables (each a pair of reals). The one-dimensional array **PLN** is also passed to **FL22**, where it is delinearized as a two-dimensional array. Linearization and delinearization of arrays are particularly common, as arrays are often linearized to provide long vectors for a vector architecture.

Array reshaping is integrated into our array analysis framework. New constraints are added to a reshaped array’s summaries to transform the summary of the formal parameter to one describing the actual parameter. When the formal parameter’s dimension sizes are projected away from this system, the result is a description of the new shape.

4 System Status and Conclusions

We have implemented all the analyses described in this paper and have been evaluating the effectiveness of this system at locating coarse-grain parallelism in scientific Fortran codes from the Perfect, SPEC and NAS benchmark suites.

Some studies of earlier interprocedural systems have shown reasonable success on linear algebra libraries [2, 3, 5, 6], but the results on larger programs have been much less promising [3]. Our system has been significantly more effective at locating parallel loops in full scientific applications, because it integrates a comprehensive suite of interprocedural analyses. As we showed with **spec77**, a combination of analyses is often required in order to parallelize an outer, coarse-grain loop.

We have also compared our results on the Perfect benchmark programs to hand-parallelized versions obtained from the University of Illinois. Our system currently correctly parallelizes approximately a fourth of these loops, either automatically or with minor code changes (such as eliminating returns on error conditions). As our system matures, we will probably parallelize at most half of these loops. There are two reasons for this. First, to parallelize some loops requires knowledge about the input data set, which is unavailable at compile time. Second, for some loops, although the information is statically available in the program, to extract the parallelism automatically does not appear to be feasible (for example, determining the value of array expressions used within subscripts of other arrays).

Our results support that an automatic parallelizer is a powerful parallel programming tool. Where static analysis is not sufficient, we need mechanisms to fill in statically unavailable or unanalyzable information, suggesting two possibilities. First, the compiler could insert code executed at run time that tests whether a loop can be executed in parallel. Second, the system could present its results to the programmer and obtain assertions about the values of variables.

Acknowledgements. The authors wish to thank Monica Lam, Patrick Sathyanathan, Shi-Wei Liao, and Alex Seibulescu for their contributions to the design and implementation of this system, and the rest of the SUIF group, particularly Chris Wilson, for providing support and infrastructure upon which this system is built.

References

- [1] M. W. Hall, J. Mellor-Crummey, A. Carle, and R. Rodriguez. FIAT: A framework for interprocedural analysis and transformation. In *Proceedings of the Sixth Workshop on Languages and Compilers for Parallel Computing*, Portland, OR, August 1993.
- [2] P. Havlak and K. Kennedy. An implementation of interprocedural bounded regular section analysis. *IEEE Transactions on Parallel and Distributed Systems*, 2(3):350–360, July 1991.
- [3] M. Hind, P. Carini, M. Burke, and S. Midkiff. Interprocedural array analysis: how much precision do we need? In *Proceedings of the 3rd Workshop on Compilers for Parallel Computers, vol. 2*, University of Vienna, Austria, July 1992.
- [4] J. Kam and J. Ullman. Global data flow analysis and iterative algorithms. *Journal of the ACM*, 23(1):159–171, January 1976.
- [5] Z. Li and P. Yew. Efficient interprocedural analysis for program restructuring for parallel programs. In *Proceedings of the ACM SIGPLAN Symposium on Parallel Programming: Experience with Applications, Languages, and Systems (PPEALS)*, New Haven, CT, July 1988.
- [6] R. Triolet, F. Irigoin, and P. Feautrier. Direct parallelization of call statements. In *Proceedings of the SIGPLAN '86 Symposium on Compiler Construction*, SIGPLAN Notices 21(7), pages 176–185. ACM, July 1986.